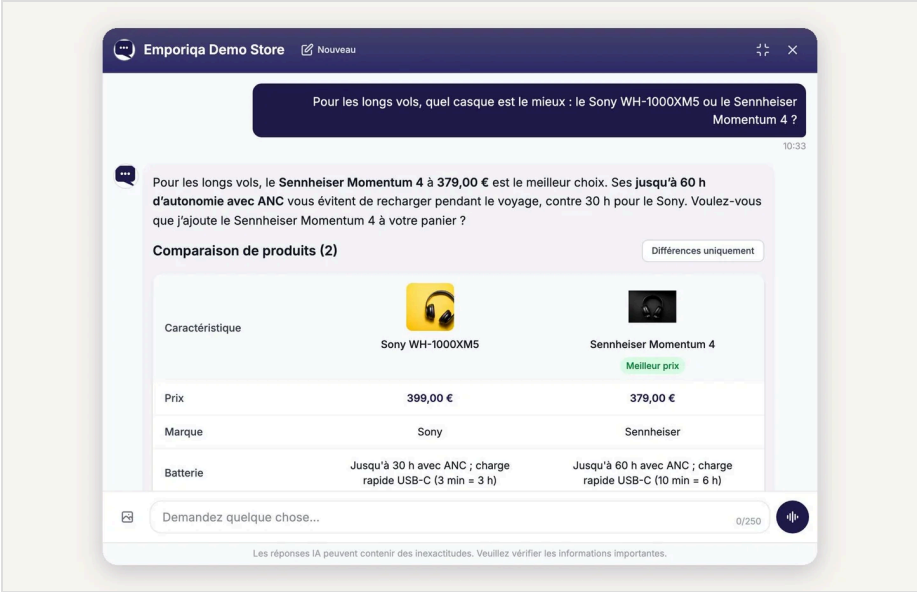
Emporiqa : Chatbot IA pour PrestaShop

Emporiqa pour PrestaShop 8.1+ et 9 est un assistant de boutique IA pour la vente et le service client, dans le chat de votre boutique et dans vos e-mails clients. Il travaille aux côtés de votre équipe, qui peut reprendre la main sur n'importe quel chat. Avant l'achat, le client décrit ce qu'il cherche ou envoie une photo, et le chat trouve les produits correspondants dans votre catalogue, les compare et les ajoute au panier. Après l'achat, il répond aux questions à partir de vos pages CMS et indique au client où en est sa commande. Un client connecté voit ses propres prix, tels que votre boutique les calcule, et est salué par son prénom. Le chat répond en 65 langues. E-mails clients : connectez la boîte mail de votre boutique (Gmail, Google Workspace, Microsoft 365, Outlook.com, IONOS, Strato, OVHcloud ou toute boîte IMAP) et Emporiqa rédige chaque réponse à partir de votre catalogue et de vos pages, dans la langue du client. Les détails de commande dans les brouillons, ainsi que les réponses automatiques, sont disponibles avec Gmail, Google Workspace et Microsoft (Outlook.com et Microsoft 365). Les réponses automatiques restent désactivées tant que vous ne les activez pas. Elles ne couvrent que les sujets pour lesquels votre équipe a déjà envoyé de nombreuses fois le brouillon sans le modifier, et chacune reste 10 minutes en attente, le temps que n'importe qui puisse l'arrêter. Toute autre réponse est un brouillon que votre équipe vérifie et envoie depuis votre propre adresse. Emporiqa ne rembourse, n'annule et ne modifie jamais une commande. Ce module synchronise votre catalogue produits et vos pages CMS avec Emporiqa, insère le widget de chat sur votre vitrine et répond aux demandes du chat concernant le panier et le statut des commandes.



- **Présentation de l'intégration** : emporiqa.com/fr/integrations/prestashop/
- **Documentation complète** : emporiqa.com/fr/docs/prestashop/ (référence du format webhook, exemples de hooks, dépannage)
- **Fonctionnalités** : emporiqa.com/fr/features/ · **FAQ** : emporiqa.com/fr/faq/ · **Tarifs** : emporiqa.com/fr/pricing/
- **Démo en ligne** : demo.emporiqa.com et une vidéo de 30 secondes. Cette démo vend de l'électronique ; sur votre boutique, le chat répond à partir de votre propre catalogue, quoi que vous vendiez. Sur une boutique de cosmétiques, « une crème hydratante sans parfum pour peau sensible » ne fait ressortir que les crèmes que votre catalogue indique sans parfum.

Prérequis

- PrestaShop 8.1+ ou 9.x (testé de la 8.1 à la 9.2)
- PHP 8.0+ sur PrestaShop 8.1, PHP 8.1+ sur PrestaShop 9. PrestaShop 8.1 fonctionne aussi sous PHP 7.x, ce module non
- Un compte Emporiqa. Aucune carte bancaire demandée, et 25 $ de crédit (environ 100 conversations) appliqué automatiquement à l'ouverture du compte

Installation

1. Téléchargez le module sur PrestaShop Addons (payant) ou sur GitHub (gratuit).
2. Dans votre back office PrestaShop, allez dans **Modules > Gestionnaire de modules > Téléverser un module** et envoyez `emporiqa.zip`.
3. Cliquez sur **Configurer** sur le module Emporiqa.
4. Cliquez sur **Se connecter à Emporiqa**. Un nouvel onglet s'ouvre sur emporiqa.com. Créez un compte gratuit (sans carte, 25 $ de crédit à l'inscription) ou connectez-vous si vous en avez déjà un, puis choisissez la boutique à connecter (ou créez-en une nouvelle). Le module est connecté à votre retour.
5. Dans l'onglet **Synchronisation**, cliquez sur **Envoyer mon catalogue** et gardez l'onglet ouvert jusqu'à ce que la barre de progression soit pleine. Produits, pages et déclinaisons sont envoyés.
6. Une fois la synchronisation terminée, testez le chat dans votre tableau de bord Emporiqa et cliquez sur « Go live » (le tableau de bord est en anglais). Jusque-là, vos clients ne voient rien.

Configurez-le depuis Claude ou ChatGPT : connectez l'un ou l'autre à Emporiqa, et il crée votre boutique, règle le chat et vous guide pendant l'installation (emporiqa.com/docs/ai-assistants/). Il vous reste à installer le module et à cliquer vous-même sur « Go live ».

Site en HTTP, ou vous préférez coller les identifiants vous-même ? Dépliez **Modifier les identifiants manuellement** sur la page Configurer. Collez le **Store ID** et le **Connection Secret** affichés dans votre tableau de bord Emporiqa, sous **Settings → Integration** (le tableau de bord est en anglais). Les deux chemins mènent au même résultat.

Statut de commande dans le chat. Une fois le module connecté, la page Configurer affiche une section **Règles prêtes à l'emploi**. Cliquez sur **Ouvrir dans Emporiqa** à côté de **Statut de commande**, puis, dans Emporiqa, cliquez sur **Try it** pour tester la règle et sur **Go live** pour l'activer (le tableau de bord est en anglais). Vous n'avez rien à copier : Emporiqa renseigne lui-même l'adresse de votre boutique lors de la connexion en un clic. Si jamais il vous la demande, le champ **Adresse du statut de commande** de la même section affiche celle à utiliser, avec un bouton **Copier**. La règle répond à « Où est ma commande ? » à partir de vos commandes PrestaShop : un client connecté à votre boutique n'indique que la référence de commande ; un invité indique aussi l'e-mail de la commande. Une fois la commande vérifiée, le chat peut donner au client tout ce

que montre sa page de commande : statut, suivi, articles et leurs prix, totaux, paiement, transporteur et délai de livraison, adresses de livraison et de facturation. Quand le client est connecté, Emporiqa peut aussi demander à votre boutique qui il est : le nom et l'e-mail de son compte et ses 10 commandes les plus récentes (numéro, date, statut, total) dans les boutiques synchronisées, pour que le chat réponde à « Où est ma commande ? » avec sa dernière commande sans demander de numéro, et préremplisse son e-mail dans vos règles. Seuls le compte et les commandes du client connecté sont envoyés, jamais ceux d'un autre client, ni une commande passée en invité, ni une adresse. Les autres règles prêtes à l'emploi d'Emporiqa (**Settings > Rules**) transmettent par e-mail à votre équipe les demandes de retour, d'annulation, de modification de commande, de devis et de facture. Si vous vous êtes connecté avec des identifiants saisis à la main et que la section n'apparaît pas encore, cliquez sur **Tester la connexion** dans l'onglet Synchronisation. Pour les boutiques qui utilisent déjà l'ancienne adresse de suivi de commande, celle-ci continue de fonctionner comme avant ; une fois la règle Statut de commande activée, la note sous **Avancé** explique comment désactiver l'ancien suivi.

Configuration

Tous les paramètres sont gérés depuis la page de configuration du module (**Modules > Emporiqa > Configurer**) :

Connexion

La méthode recommandée est **Se connecter à Emporiqa** (échange signé en un clic, aucun identifiant à coller). Sur les sites en HTTP, ou pour une configuration manuelle, dépliez **Modifier les identifiants manuellement** :

Paramètre	Description	Par défaut
Store ID	Votre identifiant de boutique Emporiqa (rempli automatiquement par la connexion en un clic)	(aucun)
Connection Secret	Secret de signature HMAC-SHA256 (rempli automatiquement par la connexion en un clic)	(aucun)

Boutiques et langues

Paramètre	Description	Par défaut
Boutiques (multiboutique uniquement)	Boutiques dont le catalogue est synchronisé et où le chat s'affiche. Toutes les boutiques partagent une seule boutique Emporiqa ; chaque boutique y est un canal	Toutes les boutiques actives
Langues	Langues incluses dans les données synchronisées ; leurs pages affichent le chat. Les pages dans une langue non cochée n'affichent pas le chat	Toutes les langues actives de la boutique

Règles prêtes à l'emploi

Paramètre	Description	Par défaut
Adresse du statut de commande	L'adresse qu'Emporiqa appelle pour la règle Statut de commande. Emporiqa la renseigne lui-même ; ne la copiez que s'il vous la demande	générée automatiquement
Suivi de commande	L'ancienne façon de répondre à « Où est ma commande ? », remplacée par la règle Statut de commande. Il continue de fonctionner pour les boutiques qui l'utilisent déjà, et passe sous **Avancé** comme *Ancien suivi de commande* dès que les règles prêtes à l'emploi s'affichent	Activé

Mettez le chat sur vos pages (section affichée une fois le module connecté, repliée par défaut)

Trois extraits avec un bouton **Copier** : un lien de question (`<a href="#emporiqa=Est-il imperméable ?" class="emporiqa-ask">Est-il imperméable ?</a>`), un lien qui ouvre le chat (`<a href="#emporiqa" class="emporiqa-ask">Discutez avec nous</a>`) et une page de chat (`<div class="emporiqa-inline"></div>`, le chat intégré à la page, et non dans le coin de l'écran). Collez-les dans une page CMS (**Apparence > Pages**) ou une description de produit, dans la vue **Code source** de l'éditeur : PrestaShop conserve ces extraits basés sur des classes et supprime les attributs data-. Un lien de question envoie sa question tout de suite quand votre boutique figure sous **Site lock** dans Emporiqa (**Settings > Security**) ; sinon, la question est préremplie et le client n'a plus qu'à l'envoyer. Le module n'ajoute rien de lui-même à votre boutique. Plus d'options : emporiqa.com/docs/widget-embedding/#open-from-page (en anglais).

Avancé

Paramètre	Description	Par défaut
Synchroniser automatiquement les produits	Envoie chaque modification de produit dès son enregistrement	Activé
Synchroniser automatiquement les pages	Envoie chaque modification de page CMS dès son enregistrement	Activé
URL du webhook	Adresse Emporiqa à laquelle la boutique envoie ses données	`https://emporiqa.com/webhooks/sync/`
Taille des lots	Produits ou pages envoyés par requête lors d'une synchronisation depuis l'onglet Synchronisation	25

Les opérations de panier dans le chat sont toujours activées. Aucune configuration nécessaire.

Mention de l'IA

Le message d'accueil par défaut indique au client qu'il s'adresse à l'assistant IA de la boutique, dans chacune des langues que le chat parle. Un message d'accueil personnalisé doit conserver cette mention ; s'il l'omet, il est refusé à l'enregistrement. La section 8.6 des conditions d'utilisation d'Emporiqa considère la suppression de cette mention, y compris par du CSS ou du code personnalisé, comme un manquement au contrat.

Garder votre catalogue à jour

Le module envoie automatiquement à Emporiqa les modifications de produits, de pages et de commandes, au fil de l'eau, via les hooks PrestaShop. Celles qui ne touchent qu'un produit, promotion ou autre prix spécifique (SpecificPrice), modification d'une règle de prix catalogue, changement d'image ou de déclinaison, déclenchent d'elles-mêmes un nouvel envoi du produit concerné. Une promotion datée est renvoyée quand elle commence et quand elle se termine : les visites de la boutique le vérifient au plus toutes les 15 minutes, jusqu'à 100 produits par visite, du plus ancien changement au plus récent, la visite suivante reprenant la suite. Une règle de prix catalogue qui couvre plus de 100 produits inscrit à la place une demande de synchronisation manuelle dans le journal ; un simple

changement de stock ou une mise en rupture envoie une mise à jour compacte, limitée à la disponibilité, sans reconstruire le produit entier.

Certains changements touchent l'ensemble du catalogue (renommage de catégories ou de marques, rafraîchissement des taux de change, modification de taux de TVA ou de groupes de règles de TVA, modification de règles panier, nouvelle langue activée). Relancer depuis ces hooks une synchronisation produit par produit, de façon synchrone, bloquerait la requête admin ; le module se contente donc d'enregistrer un avertissement dans **Paramètres avancés** → **Journaux** et laisse le rafraîchissement du catalogue à un lancement manuel.

Relancez une synchronisation complète (**Tout synchroniser**) depuis l'onglet **Synchronisation** quand :

- Vous voyez l'un des avertissements de « changement à l'échelle du catalogue » dans le journal PrestaShop
- Vous ajoutez une nouvelle boutique en mode multi-boutique (les produits existants ne porteront pas les données de cette boutique tant qu'une autre opération ne les aura pas modifiés)
- Vous importez des produits en masse depuis un fichier CSV (PrestaShop contourne parfois les hooks d'enregistrement standards lors des imports en masse)
- Un script personnalisé, une migration ou un autre module écrit des données catalogue directement en base
- Emporiqa a été injoignable pendant une période prolongée (panne réseau, maintenance planifiée, identifiants expirés)

Par sécurité, lancez une synchronisation complète une fois par semaine, pour rattraper une éventuelle dérive due à un envoi en arrière-plan qui aurait échoué.

Champs du payload produit

Au-delà des champs présentés dans la référence du payload webhook, le payload complet du produit et de ses déclinaisons porte ces champs natifs PrestaShop, de merchandising et de prix :

- `condition` : chaîne ou null ; la `condition` du produit PrestaShop (`"new"`, `"used"` ou `"refurbished"`).
- `is_virtual` : booléen ; vrai pour les produits dématérialisés sans expédition.
- `available_for_order` : booléen ; faux pour les produits en mode catalogue, en affichage seul. L'assistant les décrit toujours, mais ne les ajoute pas au panier.

- `max_order_quantities` : dictionnaire par canal (`{canal: int|null}`) de la quantité maximale autorisée par commande. PrestaShop n'ayant pas de maximum natif par commande, ce champ vaut pour l'instant toujours `null` (aucune limite). Il est là pour la parité de contrat entre plateformes, afin qu'une source personnalisée puisse le renseigner plus tard.
- `tier_prices` : liste par devise des remises sur quantité et des tarifs dégressifs (`[{min_quantity, price}]`), présente sur une entrée de prix uniquement si le produit ou la déclinaison a des remises sur quantité configurées dans PrestaShop. Chaque palier reprend le prix unitaire du visiteur non connecté à ce seuil, avec la précision que le panier multiplie (6 décimales, ou au centime quand la boutique arrondit chaque article), pour que le prix d'une quantité corresponde au total du panier. Les paliers réservés à un groupe, à un client ou à un pays (B2B) sont volontairement exclus.

Ces champs font partie du payload complet produit et déclinaison, pas de l'événement léger `product.availability`, qui ne porte que le numéro d'identification, le SKU, les statuts de disponibilité par canal et les quantités en stock.

Une déclinaison ne porte pas `descriptions`, `categories`, `brands`, `variation_attributes` ni `is_parent` : Emporiqa reprend les trois premiers de son produit et les ignore sur une déclinaison.

Un produit est envoyé pour les boutiques où il est actif et n'est pas masqué avec **Visibilité : Nulle part** ; passer un produit à Nulle part le retire d'Emporiqa. En multiboutique, une déclinaison n'est envoyée que pour les boutiques qui la vendent, et le stock du produit dans une boutique ne compte que ces déclinaisons. Les `links` sont l'URL du produit dans chaque boutique et chaque langue, construite comme votre boutique la construit (avec la catégorie, quand votre format d'URL produit en a une).

Structure du module

```
emporiqa/
├── emporiqa.php                # Classe principale du module (hooks, install, config)
├── config.xml                  # Métadonnées du module
├── logo.png                    # Icône du module
├── classes/
│   ├── EmporiqaActionEndpoint.php    # Endpoint des règles prêtes à l'emploi (order_status,
customer_prices, customer_info, verify)
│   ├── EmporiqaCartApiEndpoint.php   # Corps de l'API panier (POST uniquement, jamais en cache)
│   ├── EmporiqaConnectHandshake.php  # Corps de la connexion en un clic
│   ├── EmporiqaJsonResponse.php      # L'unique helper de réponse JSON (lisible par PHP 7)
│   ├── EmporiqaCartHandler.php       # Opérations de panier dans le chat
│   ├── EmporiqaChannelResolver.php   # Mappage multi-boutique → canal
│   ├── EmporiqaConnectNonce.php      # Stockage du vérificateur PKCE de la connexion en un clic
│   ├── EmporiqaCustomerPrices.php    # Ce que paie un client connecté (customer_prices)
│   ├── EmporiqaCustomerInfo.php      # Qui est un client connecté et ses dernières commandes
```

```
(customer_info)
│   ├── EmporiqaLanguageHelper.php    # Utilitaires de mappage des langues
│   ├── EmporiqaOrderFormatter.php    # Formatage du payload commande
│   ├── EmporiqaOrderStatus.php       # Recherche du statut de commande, dédoublonnage et limite de débit
│   ├── EmporiqaOrderTrackingEndpoint.php # Corps du suivi de commande (même limite de débit
qu'order_status)
│   ├── EmporiqaPageFormatter.php     # Formatage du payload page CMS
│   ├── EmporiqaProductFormatter.php  # Formatage du payload produit/déclinaison
│   ├── EmporiqaSchema.php            # Tables du module, créées et réparées à l'installation et à la mise
à jour
│   ├── EmporiqaSignatureHelper.php   # Signature et vérification HMAC-SHA256
│   ├── EmporiqaSyncService.php       # Orchestration de la sync en masse
│   ├── EmporiqaTokenEndpoint.php     # Jeton client signé pour le widget
│   └── EmporiqaWebhookClient.php     # Client HTTP pour la livraison des webhooks
├── controllers/
│   ├── admin/
│   │   ├── AdminEmporiqaController.php        # Redirection onglet menu admin
│   │   └── AdminEmporiqaConnectController.php # Handshake de connexion en un clic
│   └── front/
│       ├── action.php                # Endpoint des règles prêtes à l'emploi (/module/emporiqa/action)
│       ├── cartapi.php               # Endpoint API panier (/module/emporiqa/cartapi)
│       ├── ordertracking.php         # Endpoint de suivi de commande (/module/emporiqa/ordertracking)
│       └── token.php                 # Endpoint du jeton client (/module/emporiqa/token)
├── views/
│   ├── css/admin.css                 # Styles de configuration admin
│   ├── img/                          # Images du module (logo rectangulaire)
│   ├── js/
│   │   ├── admin-sync.js            # UI de sync en masse avec suivi de progression
│   │   ├── front-cart-handler.js    # Intégration panier du widget chat
│   │   └── front-customer-token.js  # Transmet le jeton client au widget
│   └── templates/
│       ├── admin/configure.tpl       # Template de la page de configuration
│       ├── admin/sync_tab.tpl        # Template de l'onglet synchronisation
│       ├── admin/sync_health_line.tpl # Ligne « dernière synchronisation complète » de l'onglet
│       └── hook/header.tpl           # Embed du widget (hook displayHeader)
├── translations/fr.php               # Traduction française du back office
└── upgrade/                          # Scripts de mise à jour de version
```

Hooks PrestaShop enregistrés

Hook	Fonction
`displayHeader`	Intègre le widget de chat sur la boutique ; au plus toutes les 15 minutes, renvoie les produits dont une promotion datée a commencé ou s'est terminée
`actionProductSave`	Synchronise le produit à la création/modification
`actionProductDelete`	Envoie l'événement de suppression pour le produit et ses variations
`actionObjectCombination{Add,Update,Delete}After`	Synchronise le produit parent quand les déclinaisons changent
`actionObjectCms{Add,Update,Delete}After`	Synchronise les pages CMS à la création/modification/suppression
`actionValidateOrder`	Capture l'ID de session chat et envoie l'événement order.completed

Hook	Fonction
`actionOrderStatusPostUpdate`	Envoie order.completed pour les captures de paiement tardives
`actionUpdateQuantity`	Émet un événement léger `product.availability` quand le stock change (sans reconstruction complète du produit)
`actionProductOutOfStock`	Émet un événement `product.availability` lors des transitions de seuil de stock
`actionObjectSpecificPrice{Add,Update,Delete}After`	Re-synchronise le produit concerné quand un prix spécifique (promo, réduction par groupe, remise sur quantité) est créé, modifié ou supprimé
`actionObjectSpecificPriceRule{Update,Delete}Before`	Re-synchronise les produits couverts par une règle de prix catalogue, pour qu'une règle supprimée ou restreinte n'affiche plus sa remise
`actionAdminSpecificPriceRuleController{Delete,Bulkdelete}Before`	Re-synchronise ces produits quand une règle est supprimée depuis la page Règles de prix catalogue, avant que ses lignes ne disparaissent
`actionObjectImage{Add,Update,Delete}After`	Re-synchronise le produit concerné quand ses images changent
`actionObjectCategory{Update,Delete}After`	Enregistre un avertissement pour que le marchand lance une synchronisation complète (impact à l'échelle du catalogue)
`actionObjectManufacturer{Update,Delete}After`	Enregistre un avertissement pour que le marchand lance une synchronisation complète (impact à l'échelle du catalogue)
`actionObjectCartRule{Add,Update,Delete}After`	Enregistre un avertissement pour que le marchand lance une synchronisation complète (impact à l'échelle du catalogue)
`actionObjectCurrencyUpdateAfter`	Enregistre un avertissement pour que le marchand lance une synchronisation complète (impact prix à l'échelle du catalogue)
`actionObjectTaxUpdateAfter` / `actionObjectTaxRulesGroupUpdateAfter`	Enregistre un avertissement pour que le marchand lance une synchronisation complète (impact prix à l'échelle du catalogue)
`actionObjectLanguageAddAfter`	Enregistre un avertissement pour que le marchand lance une synchronisation complète (nouvelle locale à compléter)

Hooks d'extensibilité

Les développeurs peuvent se brancher sur le pipeline de sync pour personnaliser les payloads ou annuler des syncs :

Hook	Fonction	Paramètres clés
`actionEmporiqaFormatProduct`	Modifier le payload produit/variation avant envoi	`&$data`, `$product`, `$event_type`
`actionEmporiqaFormatPage`	Modifier le payload page avant envoi	`&$data`, `$page`, `$event_type`
`actionEmporiqaFormatOrder`	Modifier le payload de l'événement `order.completed`	`&$data`, `$order`
`actionEmporiqaShouldSyncProduct`	Annuler conditionnellement une sync produit	`$product`, `$event_type`, `&$should_sync`
`actionEmporiqaShouldSyncPage`	Annuler conditionnellement une sync page	`$page`, `$event_type`, `&$should_sync`
`actionEmporiqaWidgetParams`	Modifier les paramètres d'intégration du widget de chat	`&$params`
`actionEmporiqaOrderStatus`	Modifier la réponse de la règle Statut de commande, ou ajouter vos propres champs sous `extra`	`&$data`, `$order`
`actionEmporiqaOrderTracking`	Modifier la réponse du suivi de commande	`&$data`, `$order`
`actionEmporiqaCustomerInfo`	Modifier ce qu'Emporiqa apprend d'un client connecté (nom et e-mail du compte, dernières commandes), ou ajouter vos propres champs sous `extra`	`&$data`, `$customer`

`actionEmporiqaOrderStatus` s'exécute une fois la réponse remplie par le module (statut, suivi, articles, totaux, paiement, transporteur, adresses) : vous pouvez donc en modifier n'importe quelle partie. Placez vos propres champs sous `extra`, que le chat lit quand le client les demande : clés texte, valeurs texte,

nombres, booléens ou listes et objets imbriqués, 30 clés au plus, 3 niveaux et 500 caractères par texte. Les autres clés ajoutées sont ignorées.

```
public function hookActionEmporiqaOrderStatus(array $params)
{
    $order = $params['order'];
    $params['data']['extra']['gift_message'] = 'Joyeux anniversaire';
    $params['data']['extra']['warehouse'] = 'Lyon';
}
```

`actionEmporiqaCustomerInfo` s'exécute une fois `$params['data']` rempli par le module : `customer` (`name`, `first_name`, `last_name`, `email` du compte) et `orders` (10 au plus, les plus récentes d'abord : `order_number`, `placed_at`, `status_code`, `status_label`, `total`, `currency`). Retirez ce qu'Emporiqa ne doit pas savoir, ou ajoutez des champs sous `extra`, avec les mêmes limites que ci-dessus.

```
public function hookActionEmporiqaCustomerInfo(array $params)
{
    $customer = $params['customer'];
    unset($params['data']['customer']['email']);
    $params['data']['extra']['loyalty_points'] = 120;
}
```

Tarifs

Le module est payant sur PrestaShop Addons et gratuit sur GitHub. Le service Emporiqa, lui, est facturé à l'usage : 0,25 $ par conversation et 0,15 $ par réponse e-mail rédigée par l'IA et envoyée par vous, avec 25 $ de crédit offert une seule fois à l'inscription (environ 100 conversations) et aucune carte bancaire demandée. Vous fixez une limite mensuelle dans votre espace facturation (199 $ par défaut pour les nouvelles boutiques) : c'est un plafond, pas un forfait, et vous pouvez le relever, le baisser ou le supprimer. Une fois la limite atteinte, le chat est masqué sur votre boutique et la facturation s'arrête. Aucun frais par utilisateur ; les coûts des modèles d'IA sont inclus. Le mode vocal (le client pose sa question au micro et entend la réponse lue à voix haute) est en option et désactivé par défaut : une conversation où le client utilise la voix est facturée 0,15 $ de plus, une seule fois, et compte dans cette limite. Prix hors TVA. Offre Enterprise pour les catalogues de plus de 100 000 produits. Tarifs complets sur emporiqa.com/fr/pricing/.

Problèmes connus

- Les messages affichés par les boutons de **Synchronisation** et **Tester la connexion**, ainsi que les erreurs de la connexion en un clic, sont uniquement en anglais. Le reste de la page de configuration est disponible en français et en anglais.
- Les contrôleurs de `controllers/` et `emporiqa.php` gardent une syntaxe PHP 7 (pas de virgule finale dans les appels ou les paramètres), afin qu'une boutique restée en PHP 7 obtienne une erreur propre au lieu d'une erreur fatale ; le code PHP 8 se trouve dans `classes/`.
- La liste complète, avec le détail technique, se trouve dans CHANGELOG.md (en anglais).

Support

Écrivez à support@emporiqa.com. Rosen Hristov accompagne lui-même chaque boutique, avant et après la vente.

Licence

Academic Free License 3.0 (AFL-3.0)